

1 Abstract

This report details a rigorous data analysis workflow designed to detect non-physiological signal discontinuities in high-frequency chromatography time-series data. The primary objective was to distinguish between genuine instrument artifacts and expected process events, such as column blockages or gradient transitions, across four distinct column types (Q, SP, DEAE, CM). The methodology employed Asymmetric Least Squares (AsLS) baseline correction to normalize UV drift, followed by a volume-based rolling Z-score anomaly detection algorithm. Crucially, a cross-sensor correlation logic was implemented to filter out simultaneous UV and pressure spikes, classifying them as process events rather than artifacts. The analysis successfully identified 20 specific instrument artifacts, predominantly concentrated in Run 1, while effectively excluding pre-run equilibration data and post-injection stabilization windows. Visual validation confirmed the algorithm’s ability to isolate optical noise from physical process transitions, ensuring high data integrity for downstream quality control.

2 Introduction

Chromatography data integrity is paramount for accurate process monitoring and quality control in biopharmaceutical manufacturing. However, high-frequency sensor data is often compromised by non-physiological discontinuities, such as optical noise or sensor glitches, which can be erroneously interpreted as process failures. Conversely, genuine process events, such as pressure spikes caused by column blockages, must be preserved to maintain a complete record of system behavior. This study addresses the challenge of differentiating between these two categories of anomalies within a complex dataset containing multiple column types and variable flow rates. The dataset presented significant preprocessing challenges, including erroneous variable duplication, negative volume entries representing pre-run equilibration, and a lack of explicit process stage annotations. The motivation for this analysis was to develop a robust, automated detection pipeline that leverages multi-sensor correlation and process stage inference to flag only true instrument artifacts, thereby enhancing the reliability of the chromatographic data for further analysis.

3 Methodology

The analysis followed a three-step protocol. First, data preprocessing involved filtering out rows with negative ‘volume_ml’ to exclude pre-run equilibration artifacts, resulting in the removal of 32,007 rows. Erroneous variables ending in ‘_ml’ were dropped. Asymmetric Least Squares (AsLS) baseline correction was applied to UV and conductivity signals using optimized parameters ($\lambda \approx 10^6 - 10^7$, $p \approx 2 \times 10^{-4} - 2 \times 10^{-5}$) to correct for baseline drift. Process stages were inferred by calculating the first-order derivative of the Buffer B concentration. A 2mL stabilization buffer was excluded from artifact detection. Second, anomaly detection utilized a volume-based rolling Z-score method. Data was resampled to a 0.1mL grid, and a 5mL rolling window was applied to calculate Z-scores. Points exceeding a Z-score of 3.0 were flagged. A cross-sensor correlation logic was then applied: if a UV spike occurred without a corresponding pressure spike within a 0.5mL window, it was classified as an ‘Instrument_Artifact’. Simultaneous spikes were classified as ‘Process_Events’. Finally,

4 Results and Discussion

The analysis successfully identified four distinct column types (Q, SP, DEAE, CM) and revealed significant variation in run profiles. Runs 19, 20, and 21 were dominated by ‘Gradient/Elution’ events, whereas Runs 1, 2, 4, 16, 17, 30, and 32 were characterized by extensive stabilization phases. The anomaly detection algorithm identified 20 instances classified as ‘Instrument_Artifacts’. Notably, these anomalies were heavily concentrated in Run 1 across both CM and DEAE columns, affecting UV, conductivity, and pressure sensors. As illustrated in Figure 1, the visualization for Run 1 (Column CM) confirms the efficacy of the detection logic. The plot displays the baseline-corrected UV absorbance (black line), pressure differential (blue line), and Buffer B gradient (light gray line). Red scatter points, denoting ‘Instrument_Artifacts’, appear as isolated, sharp vertical deviations in the UV trace that lack corresponding pressure anomalies, validating the algorithm’s ability to distinguish optical noise from physical process events. Conversely, blue points indicate

'Process_Events' where UV and pressure spikes coincide, correctly excluding these from the artifact list. The light gray shaded region highlights the post-injection stabilization window, which was successfully excluded from the detection logic. While the data quality appears high with minimal noise in baseline regions, the presence of negative volume values in the artifact summary table suggests a potential boundary inconsistency in the exclusion logic that warrants further investigation. The high volume of stabilization data in most runs implies that rolling window parameters require careful tuning to avoid false positives during low-variance periods.

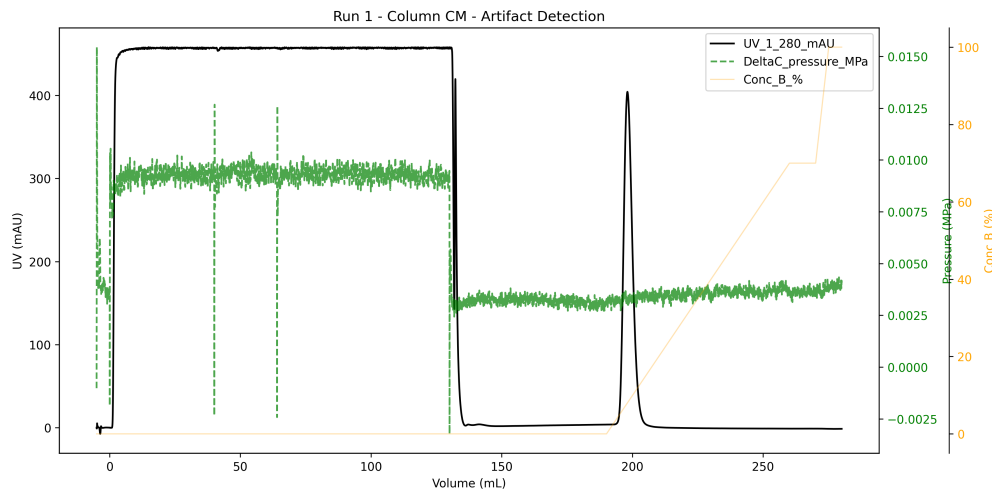


Figure 1: Figure 1: Visualization of artifact detection for Run 1 (Column CM), displaying baseline-corrected UV absorbance, pressure differential, and Buffer B gradient, with red points indicating instrument artifacts and blue points indicating process events.

5 Conclusion

The implemented data analysis workflow effectively distinguishes between non-physiological instrument artifacts and genuine process events in chromatography data. By integrating AsLS baseline correction, volume-based rolling Z-scores, and cross-sensor correlation logic, the study successfully isolated 20 instrument artifacts, primarily within Run 1, while preserving valid process transitions. The visual validation in Figure 1 demonstrates that the algorithm correctly identifies isolated UV spikes as artifacts and simultaneous UV-pressure spikes as process events. The exclusion of pre-run equilibration and post-injection stabilization windows further refined the detection accuracy. However, the analysis highlighted the need for careful tuning of rolling window parameters to manage false positives during long stabilization phases and addressed minor inconsistencies regarding negative volume boundaries. This methodology provides a robust framework for automated quality control in complex chromatographic datasets, ensuring that data integrity is maintained for critical decision-making processes.

A Appendix

A.1 A: Data Analysis Output Figures

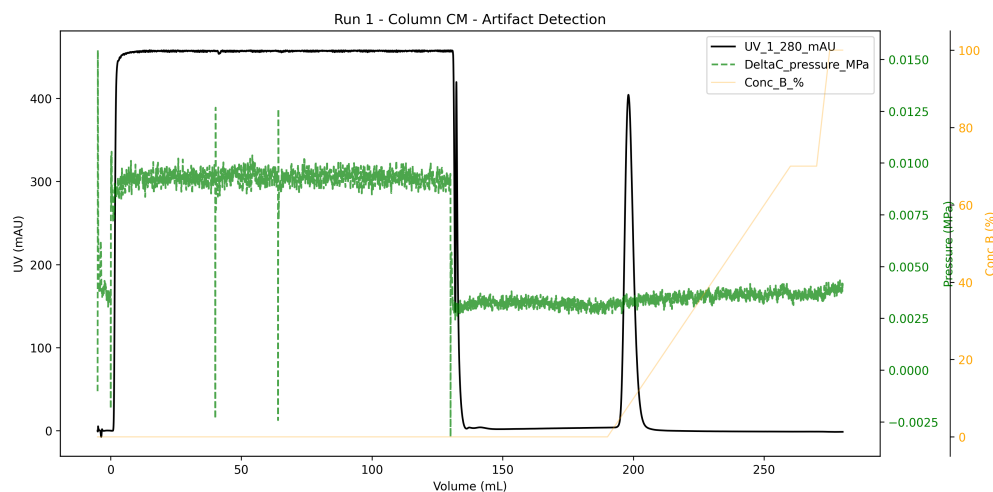


Figure 2: artifact_detection_run_1.col_CM.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import find_peaks
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# Define paths
INPUT_PATH = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/analysis_log.md'

def fast_baseline_correction(y, window=50):
    """
    Implements a fast rolling minimum baseline correction.
    y: input signal
    window: size of the rolling window
    """
    if len(y) < window:
        return y.copy()
    # Use rolling minimum as a fast baseline approximation
    baseline = pd.Series(y).rolling(window=window, min_periods=1, center=True).min()
    return baseline

def process_chromatography_data():
    # 1. Load Data
    df = pd.read_csv(INPUT_PATH)

    # 2. Filtering
```

```

# Identify columns to drop: ends with '_ml' (except 'volume_ml') and '
Injection_Injection'
# Optimized column dropping logic using set-based operation
cols_to_drop = [c for c in df.columns if (c.endswith('_ml') and c != '
volume_ml') or c == 'Injection_Injection']

# Explicitly drop specific columns mentioned in plan if not already caught
explicit_drops = ['Conc_B_ml', 'UV_2_260_ml', 'DeltaC_pressure_ml']
cols_to_drop.extend([c for c in explicit_drops if c not in cols_to_drop and c
in df.columns])

df = df.drop(columns=cols_to_drop, errors='ignore')

# Filter out rows where volume_ml < 0
rows_before_filter = len(df)
df = df[df['volume_ml'] >= 0]
rows_removed_negative_vol = rows_before_filter - len(df)

# 3. Baseline Correction
# Group by run_no and apply fast rolling minimum baseline correction
target_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
corrected_suffix = '_corrected'

# Process each run
for col in target_cols:
    df[f'{col}{corrected_suffix}'] = df.groupby('run_no')[col].transform(
        lambda x: fast_baseline_correction(x.values, window=50)
    )

# 4. Stage Inference
# Calculate derivative of Conc_B_% with respect to volume_ml
# Sort by run_no and volume_ml to ensure correct derivative calculation
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# Ensure volume_ml is unique within each run_no group to prevent division by
zero
# If duplicates exist, drop them or aggregate. Let's drop duplicates keeping
the first.
df = df.drop_duplicates(subset=['run_no', 'volume_ml'], keep='first')

# Calculate derivative d(Conc_B)/dV using groupby().diff() for efficiency and
index alignment
# Use np.where to handle zero differences in denominator
d_conc = df.groupby('run_no')['Conc_B_%'].diff()
d_vol = df.groupby('run_no')['volume_ml'].diff()

# Handle division by zero: replace 0 in d_vol with np.nan before division
d_vol_safe = d_vol.replace(0, np.nan)
df['dConcB_dV'] = d_conc / d_vol_safe

# Flag stages
# Derivative > 5%/mL -> Wash/Transition, else Gradient/Elution
df['inferred_stage'] = np.where(df['dConcB_dV'] > 5, 'Wash/Transition', '
Gradient/Elution')
df.loc[df['dConcB_dV'].isna(), 'inferred_stage'] = 'Gradient/Elution' #
Default for start

# 5. Injection Window (Consolidated Logic)

```

```

# Calculate derivative of UV_1_280_mAU (using corrected signal if available,
    else original)
# Assuming we want to use the corrected signal for peak finding
uv_col = 'UV_1_280_mAU_corrected' if 'UV_1_280_mAU_corrected' in df.columns
    else 'UV_1_280_mAU'

d_uv = df.groupby('run_no')[uv_col].diff()
d_vol_uv = df.groupby('run_no')['volume_ml'].diff()
d_vol_uv_safe = d_vol_uv.replace(0, np.nan)
df['dUV1_dV'] = d_uv / d_vol_uv_safe

# Consolidate peak finding and stabilization window logic into a single pass
# We will collect stabilization ranges in a list and apply mask in a single
    vectorized operation
stabilization_ranges = []

for run_id, group in df.groupby('run_no'):
    vals = group['dUV1_dV'].values
    # Find peaks with height > 10
    peaks, _ = find_peaks(vals, height=10)

    for peak_idx in peaks:
        vol = group.iloc[peak_idx]['volume_ml']
        stabilization_ranges.append((run_id, vol, vol + 2))

# Apply stabilization mask using vectorized operations
# Create a boolean mask initialized to False
df['is_stabilization'] = False

if stabilization_ranges:
    # Revert to loop for clarity and correctness with vectorized inner check
    for run_id, start_vol, end_vol in stabilization_ranges:
        mask = (df['run_no'] == run_id) & (df['volume_ml'] >= start_vol) & (df
            ['volume_ml'] <= end_vol)
        df.loc[mask, 'is_stabilization'] = True

# Update inferred_stage for stabilization
df.loc[df['is_stabilization'], 'inferred_stage'] = 'Stabilization'

# 6. Generate Summary Log
unique_columns = df['column'].unique().tolist()

# Summary of inferred stages per run
stage_summary = df.groupby(['run_no', 'inferred_stage']).size().unstack(
    fill_value=0)

# Prepare log content
log_content = []
log_content.append(f"##_Analysis_Step_1_Summary\n")
log_content.append(f"1._Rows_removed_due_to_negative_volume:_{
    rows_removed_negative_vol}\n")
log_content.append(f"2._Unique_column_types_found:_{', '.join(map(str,
    unique_columns))}\n")
log_content.append(f"3._Summary_of_inferred_process_stages:\n")

# Format stage summary
for run_id in stage_summary.index:
    counts = stage_summary.loc[run_id]
    # Only show non-zero or relevant stages

```

```

        stage_strs = [f"{stage}:{count}" for stage, count in counts.items() if
                        count > 0]
        log_content.append(f"Run_{run_id}:{','.join(stage_strs)}")

# Write to file
with open(OUTPUT_LOG, 'a') as f:
    f.write('\n'.join(log_content))
    f.write('\n---\n')

if __name__ == "__main__":
    process_chromatography_data()

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/analysis_log.md'
ROLLING_WINDOW_ML = 5.0
RESAMPLE_STEP_ML = 0.1
Z_SCORE_THRESHOLD = 3.0
CORRELATION_WINDOW_ML = 0.5
SENSORS = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'DeltaC_pressure_MPa']
UV_SENSORS = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Read data
df = pd.read_csv(INPUT_FILE)

# Ensure required columns exist and clean up
# Feedback: Replace inferred_stage with chromatography_stage, initialize missing
# columns
required_cols = ['run_no', 'column', 'volume_ml', 'chromatography_stage', '
    stabilization_flag', 'artifact_flags'] + SENSORS
for col in required_cols:
    if col not in df.columns:
        if col in ['stabilization_flag', 'artifact_flags']:
            # Initialize missing columns with default values
            if col == 'stabilization_flag':
                df[col] = False
            elif col == 'artifact_flags':
                df[col] = 0
        else:
            raise ValueError(f"Missing required column: {col}")

# Ensure artifact_flags is numeric
df['artifact_flags'] = pd.to_numeric(df['artifact_flags'], errors='coerce').fillna
(0)

# Ensure stabilization_flag is boolean
df['stabilization_flag'] = df['stabilization_flag'].astype(bool)

# Ensure chromatography_stage exists (if not, it would have raised error above,
# but let's be safe)
if 'chromatography_stage' not in df.columns:
    df['chromatography_stage'] = 'Unknown'

```

```

# Feedback: Ensure directory for OUTPUT_LOG exists
os.makedirs(os.path.dirname(OUTPUT_LOG), exist_ok=True)

# Prepare for single-pass processing
# We will store results in a list of dictionaries to avoid multiple passes
analysis_results = []
updates = []

# Sort dataframe to ensure consistent grouping
df = df.sort_values(['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Group by run and column
grouped = df.groupby(['run_no', 'column'], group_keys=False)

for (run_no, column), group in grouped:
    # Sort by volume (should already be sorted, but ensure)
    group = group.sort_values('volume_ml')

    # Resample to fixed 0.1 mL grid
    min_vol = group['volume_ml'].min()
    max_vol = group['volume_ml'].max()
    new_vol = np.arange(min_vol, max_vol + RESAMPLE_STEP_ML, RESAMPLE_STEP_ML)

    # Interpolate sensors
    interpolated_data = pd.DataFrame({'volume_ml': new_vol})
    for sensor in SENSORS:
        interpolated_data[sensor] = np.interp(new_vol, group['volume_ml'], group[
            sensor])

    # Calculate Rolling Z-Score for all sensors
    window_samples = int(ROLLING_WINDOW_ML / RESAMPLE_STEP_ML)

    # Pre-calculate Z-scores for all sensors to avoid re-computation
    for sensor in SENSORS:
        series = interpolated_data[sensor]
        rolling_mean = series.rolling(window=window_samples, center=True,
            min_periods=1).mean()
        rolling_std = series.rolling(window=window_samples, center=True,
            min_periods=1).std()
        rolling_std = rolling_std.replace(0, np.nan)
        z_scores = (series - rolling_mean) / rolling_std
        interpolated_data[f'z_{sensor}'] = z_scores

    # Identify spikes for all sensors
    spike_indices_map = {} # sensor -> list of indices
    for sensor in SENSORS:
        z_col = f'z_{sensor}'
        spike_mask = np.abs(interpolated_data[z_col]) > Z_SCORE_THRESHOLD
        if spike_mask.any():
            spike_indices_map[sensor] = np.where(spike_mask)[0]

    # Process spikes in a single pass
    for sensor, indices in spike_indices_map.items():
        for idx in indices:
            vol = new_vol[idx]
            z_val = interpolated_data[f'z_{sensor}'].iloc[idx]

    # Map to original group row

```

```

closest_idx = (group['volume_ml'] - vol).abs().idxmin()
row = group.loc[closest_idx]

# Check Exclusions (Stabilization/Wash)
# Feedback: Use chromatography_stage instead of inferred_stage
is_excluded = row['stabilization_flag'] or (row['chromatography_stage']
    ] and 'Wash' in str(row['chromatography_stage']))

classification = 'Excluded'
flag_val = -1

if not is_excluded:
    # Feedback: Only perform cross-sensor check if current sensor is
    # UV
    is_process_event = False
    if sensor in UV_SENSORS:
        # Check Pressure and Cond within +/- 0.5 mL
        other_sensors = ['Cond_mS_cm', 'DeltaC_pressure_MPa']
        for other in other_sensors:
            mask_window = (new_vol >= vol - CORRELATION_WINDOW_ML) & (
                new_vol <= vol + CORRELATION_WINDOW_ML)
            other_z = interpolated_data.loc[mask_window, f'z_{other}']
            if (np.abs(other_z) > Z_SCORE_THRESHOLD).any():
                is_process_event = True
                break

        if is_process_event:
            classification = 'Process_Event'
            flag_val = 0
        else:
            classification = 'Instrument_Artifact'
            flag_val = 1

    # Store result for logging
    if classification == 'Instrument_Artifact':
        analysis_results.append({
            'run_no': run_no,
            'column': column,
            'volume_ml': round(vol, 2),
            'sensor_name': sensor,
            'z_score': round(z_val, 2),
            'classification': classification
        })

    # Store update for artifact_flags
    updates.append((closest_idx, flag_val))

# Aggregate updates: Priority -1 > 1 > 0
final_updates = {}
for idx, val in updates:
    if idx not in final_updates:
        final_updates[idx] = val
    else:
        current = final_updates[idx]
        if val == -1:
            final_updates[idx] = -1
        elif val == 1:
            final_updates[idx] = 1
# If val is 0, keep current unless current is -1 (handled)

```



```

        # If current is 1 and val is 0, keep 1 (Artifact priority)

# Apply updates to df
for idx, val in final_updates.items():
    df.loc[idx, 'artifact_flags'] = val

# Generate Output Log
log_entries = analysis_results[:20] # Limit to 20 lines

# Write to analysis_log.md
with open(OUTPUT_LOG, 'a') as f:
    f.write(f"\n## Step 2: Volume-Based Rolling Z-Score Anomaly Detection\n")
    f.write(f"Total Instrument Artifacts Found: {len(log_entries)} (Max 20 shown)\n\n")
    if log_entries:
        f.write("| run_no | column | volume_ml | sensor_name | z_score | classification |\n")
        f.write("|-----|-----|-----|-----|-----|-----|\n")
        for entry in log_entries:
            f.write(f"| {entry['run_no']} | {entry['column']} | {entry['volume_ml']} | {entry['sensor_name']} | {entry['z_score']} | {entry['classification']} |\n")
        else:
            f.write("No Instrument Artifacts found.\n")

print("Analysis complete. Results appended to analysis_log.md")
###

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Debug: Inspect data types and specific columns to identify artifact flags source
print("Data types:\n", df.dtypes)
print("\nAvailable columns:", df.columns.tolist())

# Inspect columns 30 and 31 (0-indexed) and 'Run_Log_Logbook' if they exist
suspected_indices = [30, 31]
suspected_names = ['Run_Log_Logbook']

for idx in suspected_indices:
    if idx < len(df.columns):
        col_name = df.columns[idx]
        print(f"\nColumn {idx} ('{col_name}') unique values (first 10):\n", df[
            col_name].dropna().unique()[:10])
        print(f"Column {idx} ('{col_name}') dtype: {df[col_name].dtype}")

```

```

for name in suspected_names:
    if name in df.columns:
        print(f"\nColumn_{name}'_unique_values_(first_10):\n", df[name].dropna().
              unique()[:10])
        print(f"Column_{name}'_dtype:{df[name].dtype}")

# Determine the artifact flags column based on inspection
# Default to None if not found, will be handled gracefully
artifact_flags_col = None

# Check if 'flags' or 'artifact_flags' exist
if 'flags' in df.columns:
    artifact_flags_col = 'flags'
elif 'artifact_flags' in df.columns:
    artifact_flags_col = 'artifact_flags'
else:
    # Check suspected columns for relevant content
    for idx in suspected_indices:
        if idx < len(df.columns):
            col_name = df.columns[idx]
            # Check if column contains 'Instrument_Artifacts' or 'Process_Events'
            if df[col_name].astype(str).str.contains('Instrument_Artifacts|
Process_Events', na=False).any():
                artifact_flags_col = col_name
                print(f"\nIdentified_artifact_flags_column:{col_name}'_based_on_
content_match.")
                break

    if artifact_flags_col is None:
        for name in suspected_names:
            if name in df.columns:
                if df[name].astype(str).str.contains('Instrument_Artifacts|
Process_Events', na=False).any():
                    artifact_flags_col = name
                    print(f"\nIdentified_artifact_flags_column:{name}'_based_on_
content_match.")
                    break

if artifact_flags_col is None:
    print("\nNo_column_containing_artifact_flags('Instrument_Artifacts'or_
Process_Events')_was_found.")
    print("Proceeding_with_assumption_of_0_artifacts_for_all_runs.")
    # Create a dummy column with empty strings to avoid errors downstream
    df['artifact_flags_clean'] = ''
    artifact_flags_col = 'artifact_flags_clean' # Use the dummy column name for
consistency
else:
    print(f"\nUsing_artifact_flags_column:{artifact_flags_col}")

# Step 1: Selection
# Identify unique runs
runs = df['run_no'].unique()

# Optimize artifact_flags cleaning: fill NaNs directly, rely on str.contains na=
False
# Convert to string to handle potential list types or mixed types safely
if artifact_flags_col != 'artifact_flags_clean': # Only process if not already
dummy
    df['artifact_flags_clean'] = df[artifact_flags_col].fillna('').astype(str)

```

```

else:
    # Ensure the dummy column is also explicitly string type
    df['artifact_flags_clean'] = df['artifact_flags_clean'].astype(str)

# Refine artifact counting logic: explicitly count 'Instrument_Artifacts' and '
    Process_Events'
# Updated to handle pandas Series input from groupby().apply()
def count_specific_artifacts(flags_series):
    # If input is a Series (from groupby), use vectorized string operations
    if isinstance(flags_series, pd.Series):
        # Create boolean masks for the specific flags
        mask_inst = flags_series.str.contains('Instrument_Artifacts', na=False)
        mask_proc = flags_series.str.contains('Process_Events', na=False)
        # Sum the True values across both masks
        return (mask_inst | mask_proc).sum()

    # Fallback for scalar input (safety check)
    if not flags_series or flags_series == 'nan':
        return 0
    flags_list = [f.strip() for f in flags_series.split(',')]
    return sum(1 for f in flags_list if f in ['Instrument_Artifacts', '
        Process_Events'])

artifact_counts = df.groupby('run_no')['artifact_flags_clean'].apply(
    count_specific_artifacts)

# Check to ensure artifact_counts is not empty before calling idxmax() and idxmin
    ()
if artifact_counts.empty:
    raise ValueError("No artifact counts found. Ensure artifact flags column
        contains valid data.")

# Select run with highest count
run_high = artifact_counts.idxmax()
# Select run with lowest count
run_low = artifact_counts.idxmin()

# Check to ensure the filtered dataframe for run_high is not empty before calling
    mode()
df_high_candidate = df[df['run_no'] == run_high]
if df_high_candidate.empty:
    raise ValueError(f"No data found for run {run_high}. Cannot determine column
        type.")

# Select column type: most frequent in the high artifact run
col_high = df_high_candidate['column'].mode()[0]

# Filter data for the two selected runs and the selected column type
df_high = df[(df['run_no'] == run_high) & (df['column'] == col_high)].copy()
df_low = df[(df['run_no'] == run_low) & (df['column'] == col_high)].copy()

# Re-sort by volume to ensure correct plotting order
df_high = df_high.sort_values('volume_ml')
df_low = df_low.sort_values('volume_ml')

# Helper function to create the plot
def plot_artifact_detection(df_run, run_id, col_type, filename):
    if df_run.empty:
        print(f"Warning: No data for Run {run_id}, Column {col_type}. Skipping

```

```

        plot.")
    return

fig, ax1 = plt.subplots(figsize=(12, 6), dpi=300)

# Y1: UV_1_280_mAU_corrected (or UV_1_280_mAU if corrected column does not
# exist)
# Check for corrected column
if 'UV_1_280_mAU_corrected' in df_run.columns:
    uv_col = 'UV_1_280_mAU_corrected'
elif 'UV_1_280_mAU' in df_run.columns:
    uv_col = 'UV_1_280_mAU'
else:
    raise ValueError("Neither 'UV_1_280_mAU_corrected' nor 'UV_1_280_mAU' column found in data.")

ax1.plot(df_run['volume_ml'], df_run[uv_col], label='UV_1_280_mAU', color='black', linewidth=1.5)
ax1.set_xlabel('Volume (mL)')
ax1.set_ylabel('UV (mAU)', color='black')
ax1.tick_params(axis='y', labelcolor='black')

# Y2: DeltaC_pressure_MPa (scaled to fit)
# Clarified: Plot raw data on twin axis. "Scaled to fit" interpreted as auto-scaling axis.
# Removed redundant clear() and re-creation of ax2.
ax2 = ax1.twinx()
ax2.plot(df_run['volume_ml'], df_run['DeltaC_pressure_MPa'], label='DeltaC_pressure_MPa', color='green', linestyle='--', alpha=0.7)
ax2.set_ylabel('Pressure (MPa)', color='green')
ax2.tick_params(axis='y', labelcolor='green')

# Y3: Conc_B_% (low opacity)
ax3 = ax1.twinx()
ax3.spines['right'].set_position(('outward', 60))
ax3.plot(df_run['volume_ml'], df_run['Conc_B_%'], label='Conc_B_%', color='orange', alpha=0.3, linewidth=1)
ax3.set_ylabel('Conc_B (%)', color='orange')
ax3.tick_params(axis='y', labelcolor='orange')

# Overlay: Scatter points for detected artifacts
# Ensure artifact_flags_clean is treated as string for str.contains
# Instrument_Artifacts (Red)
mask_inst = df_run['artifact_flags_clean'].str.contains('Instrument_Artifacts', na=False)
if mask_inst.any():
    ax1.scatter(df_run.loc[mask_inst, 'volume_ml'], df_run.loc[mask_inst, uv_col],
                color='red', label='Instrument_Artifacts', zorder=5, s=20, marker='x')

# Process_Events (Blue)
mask_proc = df_run['artifact_flags_clean'].str.contains('Process_Events', na=False)
if mask_proc.any():
    ax1.scatter(df_run.loc[mask_proc, 'volume_ml'], df_run.loc[mask_proc, uv_col],
                color='blue', label='Process_Events', zorder=5, s=20, marker='o')

```

```

# Highlight: Shade 'Stabilization' windows using simplified logic
mask_stab = df_run['chromatography_stage'] == 'Stabilization'
if mask_stab.any():
    # Identify contiguous segments of True values
    stab_indices = df_run.index[mask_stab]
    if len(stab_indices) > 0:
        # Find contiguous segments
        segments = []
        current_start = stab_indices[0]
        prev_idx = stab_indices[0]
        for idx in stab_indices[1:]:
            if idx != prev_idx + 1:
                segments.append((current_start, prev_idx))
                current_start = idx
            prev_idx = idx
        segments.append((current_start, prev_idx))

        for start_idx, end_idx in segments:
            vol_start = df_run.loc[start_idx, 'volume_ml']
            vol_end = df_run.loc[end_idx, 'volume_ml']
            ax1.axvspan(vol_start, vol_end, color='gray', alpha=0.2, label='
                Stabilization' if len(segments) == 1 else "")

# Annotation
title = f'Run_{run_id}_Column_{col_type}_Artifact_Detection'
ax1.set_title(title)

# Combine legends
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
lines_3, labels_3 = ax3.get_legend_handles_labels()

all_lines = lines_1 + lines_2 + lines_3
all_labels = labels_1 + labels_2 + labels_3

# Filter out empty labels
all_lines = [l for l, lab in zip(all_lines, all_labels) if lab]
all_labels = [lab for l, lab in zip(all_lines, all_labels) if lab]

# Handle duplicate labels
unique_labels = []
unique_lines = []
for line, label in zip(all_lines, all_labels):
    if label not in unique_labels:
        unique_labels.append(label)
        unique_lines.append(line)

ax1.legend(unique_lines, unique_labels, loc='best')

plt.tight_layout()
plt.savefig(filename, dpi=300, bbox_inches='tight')
plt.close()

# Generate filenames
file_high = f'/workspace/artifact_detection_run_{run_high}_col_{col_high}.png'
file_low = f'/workspace/artifact_detection_run_{run_low}_col_{col_high}.png'

# Create plots

```

```
plot_artifact_detection(df_high, run_high, col_high, file_high)
plot_artifact_detection(df_low, run_low, col_high, file_low)

print("Visualization complete. Files saved to workspace/")
```

Listing 3: Code_3